Integration Write-up: Text Encoder + Diffusion Model

IE 7615 Deep Learning for AI · Project 2, Milestone 2 · Group 02: Abdellah Faleh, Jin-Woo Hong, Samuel Tong, Tristan Lyons · Notebook *04_integration_training.ipynb*

We connected a Transformer text encoder (CLIP ViT-L/14, or BERT with our trained adapter) to the Stable Diffusion 1.5 diffusion model [1] ourselves. We loaded the five parts separately and wrote our own denoising loop that takes **text embeddings** as input. Figure 1 shows the full path from caption to image.

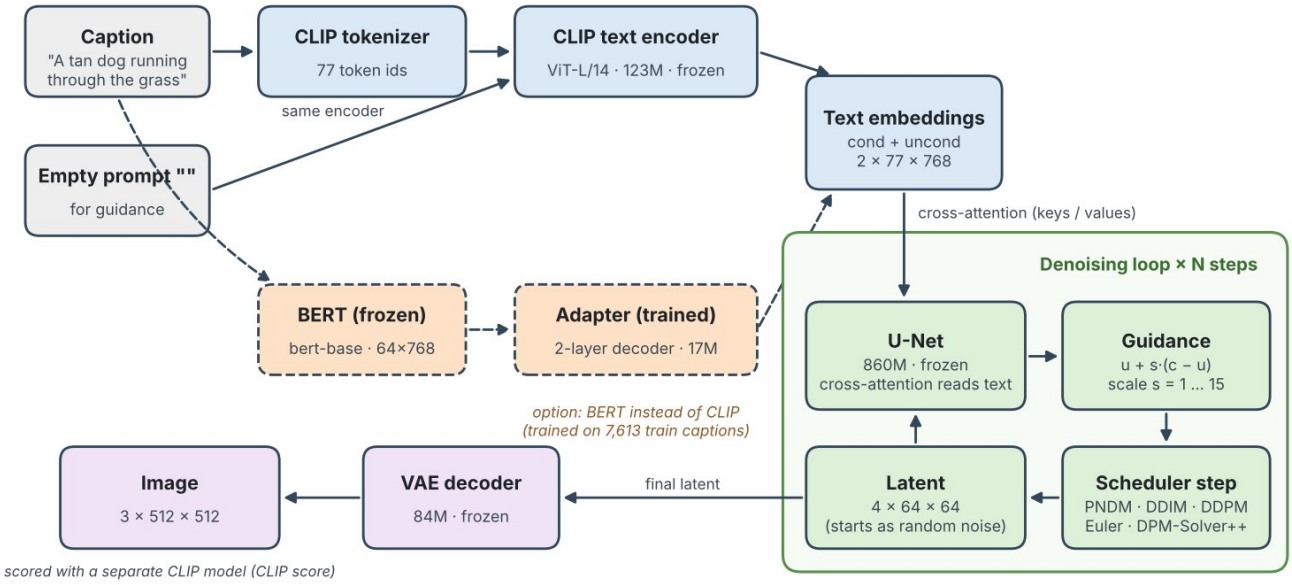


Figure 1. Architecture. Solid path: CLIP. Dashed path: BERT + our adapter. Only the adapter is trained; everything else is frozen.

1. The data flow, step by step

Shapes recorded by the notebook for the test caption "A tan dog running through the grass with the background blurry":

Step	What happens	Shape
1. Tokenize	CLIP tokenizer: 13 real tokens (start, 11 words, end), padded to 77	1 × 77 ids
2. Encode text	CLIP text encoder (123M parameters): one 768-number vector per token	1 × 77 × 768
3. Encode empty prompt	the same encoder on "" (needed for guidance)	1 × 77 × 768
4. Build the condition	empty + caption stacked together, sent to the U-Net	2 × 77 × 768
5. Start from noise	random latent from the seed (48× smaller than the image)	1 × 4 × 64 × 64
6. Predict noise	U-Net (860M) with cross-attention to the text, run on both copies	2 × 4 × 64 × 64
7. Guidance	noise = u + s · (c – u), scale s = 7.5	1 × 4 × 64 × 64
8. Scheduler step	remove that noise (PNDM by default); repeat steps 6–8 N times	1 × 4 × 64 × 64
9. Decode	divide by 0.18215, then VAE decoder (84M)	3 × 512 × 512

Table 1. Text → embedding → conditioning → image.

2. How the embedding conditions the image

The U-Net has **cross-attention** layers at several resolutions. In each one, the image features ask the questions (queries), and the 77 text vectors give the answers (keys and values). This is how a word like "dog" changes the pixels where the dog should be. The U-Net expects text vectors of size 768 (*cross_attention_dim*), the same as the CLIP encoder's hidden size; the notebook checks this. Any encoder that outputs N × 768 can be plugged in, but it must "speak the same language" as CLIP, which is why raw BERT fails and the adapter is needed (Section 3).

Classifier-free guidance [2] runs the U-Net with and without the text and pushes the result away from the "no text" prediction. A bigger scale *s* follows the text more strongly but gives less variety.

3. What we changed in the pretrained pipeline

- **Our own denoising loop.** In notebook 04 we do not call the ready-made pipeline. Our function *generate(cond, uncond, seed)* creates the noise, runs the U-Net, applies guidance, calls the scheduler and decodes the image. It takes embeddings, not text, so the text encoder can be swapped.
- **Embeddings instead of text.** In notebook 05 we compute the CLIP embeddings ourselves and pass them to the pipeline as *prompt_embeds* and *negative_prompt_embeds*. The sampler, steps, noise schedule and guidance scale are changed for the experiments.
- **A new, trained BERT → CLIP adapter.** BERT (bert-base-uncased, frozen) reads the caption (64 tokens). Our adapter has 77 learned query vectors and a 2-layer Transformer decoder (8 heads, cross-attention to the BERT tokens) and outputs 77 × 768 (17.0M parameters). It was trained to copy CLIP's embedding of the same caption (7,613 train captions, mean squared error). The U-Net and VAE were not changed.
- **Sampler diagnostics.** The loop can log, at every step, the latent's spread, the size of the text signal (c – u) and a preview of the model's current guess of the clean image.

4. Checks that the integration works

Check	Result
Our loop vs. the official pipeline (same prompt, seed 42)	mean pixel difference **0.0** / 255 (identical)
Our loop vs. the Milestone 1 image	**0.0** / 255 (identical)
Same seed, 6 different captions: does each image match its own caption best?	**6 of 6** (CLIP 27.4 right vs. 7.8 wrong)
Image from the empty prompt (same seed)	a random sketch-like pattern (CLIP 11.3 vs. the 6 captions)
Text encoder: CLIP / raw BERT / BERT + adapter (mean CLIP score)	28.8 / 9.9 / 21.4
Time and GPU memory for one image (30 steps, T4)	4.6 s, 2.8 GB

Table 2. Integration checks (notebook 04, files verify.json and conditioning_check.json).



Figure 2. The model's guess of the final image during sampling: the layout appears by step 10, details after step 20.

References. [1] R. Rombach et al., "High-resolution image synthesis with latent diffusion models," *CVPR*, 2022. [2] J. Ho and T. Salimans, "Classifier-free diffusion guidance," *NeurIPS Workshop*, 2021. [3] J. Devlin et al., "BERT," *NAACL*, 2019. [4] A. Radford et al., "CLIP," *ICML*, 2021.